Hard Link & Soft Link in Linux

In Linux, everything is treated as a **file**, and files are managed using something called an **inode**.

👉 **Inode** = A data structure that stores file metadata (permissions, owner, size, location on disk, etc.), but **NOT the file name**.

You can check inode number using:

```
ls -i <file_name>
```

1️⃣ Hard Link

👉 What is a Hard Link?

A **Hard Link** is another name for the same file (same data on disk).

It is NOT a copy.
It is NOT a shortcut.

It is simply an additional filename pointing to the **same inode (same data block)**.

Think of it like:

> One house (data) with two different names on the door.

📌 Key Characteristics of Hard Link

✅ 1. Same Inode Number

Both original file and hard link share the same inode.

```
ls -li
```

Example output:

```
12345 -rw-r--r-- 2 user user 12 Feb 16 file1.txt
12345 -rw-r--r-- 2 user user 12 Feb 16 file2.txt
```

👉 Both have inode **12345** → means same file.

✅ 2. Works Only for Files

You cannot create a hard link for directories.

❌ This will fail:

```
ln dir1 dir2
```

✅ 3. Cannot Cross Filesystems

You cannot create hard link between:

- Different file systems (ext4 → xfs)
- Different partitions
- Different devices

Example:

```
/dev/sda1 (ext4)
/dev/sdb1 (xfs)
```

Hard link between them ❌ Not possible.

✅ 4. Deleting Original File Does NOT Delete Data

Data remains available as long as **at least one hard link exists**.

Only when **all links are deleted**, the data is removed.

🔧 Hard Link Example (Step-by-Step)

```
echo "Hello Linux" > file1.txt
ln file1.txt file2.txt
```

Check inode:

```
ls -li
```

Edit file1:

```
echo "New Line" >> file1.txt
```

Check file2:

```
cat file2.txt
```

👉 You will see updated content.
Because both are same file (same inode).

🔍 Check Number of Links

Use:

```
ls -l
```

Example:

```
-rw-r--r-- 2 user user 20 Feb 16 file1.txt
```

The number **2** shows:
👉 Total hard links to that inode.

2️⃣ Soft Link (Symbolic Link)

👉 What is a Soft Link?

A **Soft Link (Symbolic Link)** is like a shortcut in Windows.

It does NOT point to data.
It points to the **file path**.

Think of it like:

> A note that says: "Go to this location to find the file."

📌 Key Characteristics of Soft Link

✅ 1. Different Inode Number

Soft link has its own inode.

```
ls -li
```

Example:

```
12345 original.txt
56789 shortcut.txt -> original.txt
```

👉 Different inode numbers.

✅ 2. Can Link Directories

You can create soft link for:

- Files ✅
- Directories ✅

Example:

```
ln -s /home/user/project project_link
```

✅ 3. Can Cross Filesystems

You can create soft link between:

- Different partitions
- Different filesystems
- Different devices

❌ 4. If Original File is Deleted → Soft Link Breaks

Example:

```
rm original.txt
cat shortcut.txt
```

Output:

```
No such file or directory
```

Because soft link only stores the path.

🔧 Soft Link Example (Step-by-Step)

```
echo "Hello World" > original.txt
ln -s original.txt shortcut.txt
```

Check inode:

```
ls -li
```

You will see:

- Different inode numbers
- Arrow symbol `->`

Example:

```
56789 shortcut.txt -> original.txt
```

📊 Hard Link vs Soft Link (Detailed Comparison)

Sr No	Hard Link	Soft Link
1	Points to actual data	Points to file path
2	Same inode as original	Different inode
3	Cannot cross filesystems	Can cross filesystems
4	Cannot link directories	Can link files & directories
5	Survives if original is deleted	Breaks if original is deleted
6	Command: `ln file1 file2`	Command: `ln -s file1 file2`

🧠 Important Concept: How Deletion Works

For Hard Link

When you delete original file:

```
rm file1.txt
```

System only removes:
👉 That filename

Data remains until:
👉 Link count becomes 0

You can check link count using:

```
ls -l
```

For Soft Link

When you delete original:

```
rm original.txt
```

Soft link becomes:
👉 Broken link (dangling link)

You can identify broken link:

```
ls -l
```

It will show in red (in many systems).

🏁 Real-World Use Cases

✅ Hard Link Use Cases

- File backup within same filesystem
- Prevent accidental deletion

- Save disk space (no duplicate data)

✅ Soft Link Use Cases

- Create shortcuts
- Link configuration files
- Point to shared libraries
- Create references across filesystems
- Manage software versions

Example:

```
ln -s /opt/app/v2/app /usr/bin/app
```

🎯 Quick Summary

Hard Link

✔ Same inode
✔ Same data
✔ No filesystem crossing
✔ Survives original deletion
❌ No directory linking

Soft Link

✔ Different inode
✔ Points to path
✔ Can cross filesystem
✔ Can link directories
❌ Breaks if original deleted